

# Project - Low Level Design

## On

### IaC Provisioning for Telecomm System

Course Name: Devops

***Institution Name:*** Medicaps University – Datagami Skill Based Course

*Student Name(s) & Enrolment Number(s):*

| Sr no | Student Name    | Enrolment Number |
|-------|-----------------|------------------|
| 1     | ANKIT YADAV     | EN22CS301136     |
| 2     | ANSHUMAN GEETE  | EN22CS301160     |
| 3     | ARTI JAIN       | EN22CS301206     |
| 4     | ATISHA JAIN     | EN22CS301235     |
| 5     | AVANI SHARMA    | EN22CS301239     |
| 6     | ISHANA HATODIYA | EN23CS3L1011     |

*Group Name:* 07D2

*Project Number:* DO-19

*Industry Mentor Name:*

*University Mentor Name:* Prof. Akshay Saxena

*Academic Year:* 2022-2026

## **Table of Contents**

### **1. Introduction.**

- 1.1. Scope of the document.
- 1.2. Intended Audience
- 1.3. System overview.

### **2. System Design.**

- 2.1. Application Design
- 2.2. Process Flow.
- 2.3. Information Flow.
- 2.4. Components Design
- 2.5. Key Design Considerations
- 2.6. API Catalogue.

### **3. References**

### **4. GitHub**

## 1. Introduction

This document presents the system design for the IaC Provisioning for Telecomm System project, developed as part of the Datagami Skill Based Course at Medicaps University. The project demonstrates the automation of infrastructure provisioning for a Telecom-grade environment using Infrastructure as Code (IaC) tools including Terraform and Ansible.

The objective is to replace manual infrastructure setup with automated, repeatable, error-free provisioning that can deploy both local Docker-based telecom service environments and cloud infrastructure (AWS). The system provisions compute instances, installs dependencies, configures telecom network components, and deploys required services using automated scripts.

### 1.1 Scope of the Document

This document covers:

- Complete IaC system architecture and provisioning workflow
- Terraform configuration for cloud resource provisioning
- Ansible playbooks for configuration management
- Docker-based local telecom environment deployment
- CI/CD automation for provisioning
- Security considerations and environment variable handling
- API catalogue (if applicable for telecom services)

Excluded:

- Internal telecom business logic
- Proprietary telecom signalling algorithms
- Network protocol handling of telecom core elements

### 1.2 Intended Audience

| Audience            | Purpose                                                   |
|---------------------|-----------------------------------------------------------|
| DevOps Engineers    | Understand IaC automation using Terraform and Ansible     |
| Cloud Architects    | Review provisioning, security, and scaling considerations |
| Telecom Engineers   | Understand automated telecom environment deployment       |
| Academic Evaluators | Technical depth and implementation analysis               |
| Future Maintainers  | Extend provisioning scripts safely                        |

A basic understanding of Terraform, Ansible, AWS, and Docker is recommended.

## 1.3 System Overview

The system consists of two decoupled repositories:

### Repository 1 — IaC Scripts (Telecomm-IaC-Provisioning)

Contains Terraform modules, Ansible playbooks, inventory files, Docker environment setup, and automation scripts.

### Repository 2 — Telecom Application Repository

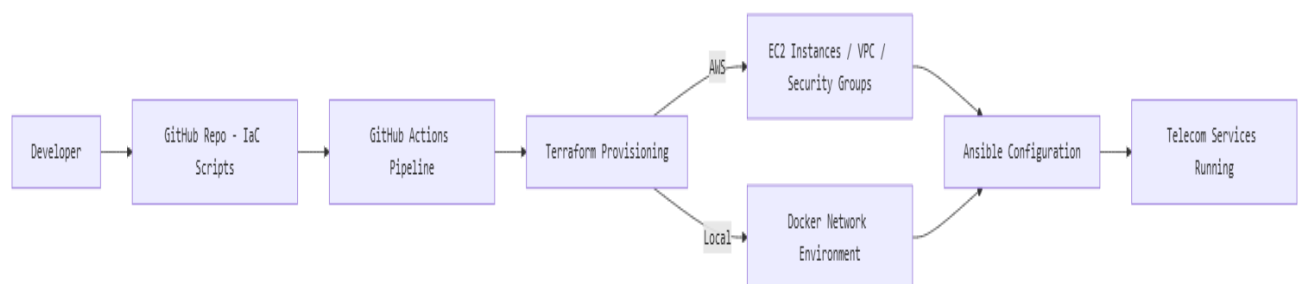
Contains telecom simulation services (e.g., Call Simulation, SMS Router, Signalling Handler, Network Registry).

### Workflow Overview

Whenever infrastructure provisioning is triggered:

1. Terraform provisions cloud infrastructure or local Docker networks.
2. Ansible configures OS, installs dependencies, deploys telecom services.
3. Docker containers simulate telecom components locally or on EC2.
4. CI/CD automates the entire provisioning pipeline.

## High-Level Architecture Diagram



## 2. System Design

### 2.1 Application Design

The system provisions two environments:

- Cloud Environment (AWS) via Terraform
- Local Docker Telecom Network via Docker Compose

## Telecom Components in Application Repo

| Module               | Purpose                    |
|----------------------|----------------------------|
| Call Handler         | Simulates call signalling  |
| SMS Router           | Routes messages            |
| Network Registry     | Stores subscriber metadata |
| Monitoring Dashboard | Streamlit/Grafana-based UI |

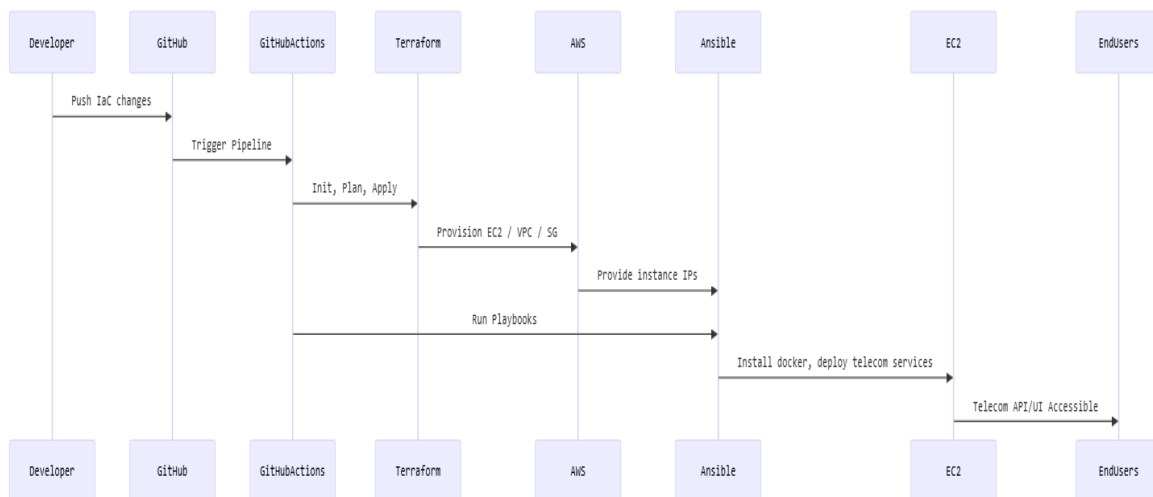
## Container Design

Each telecom component packaged into Docker images:

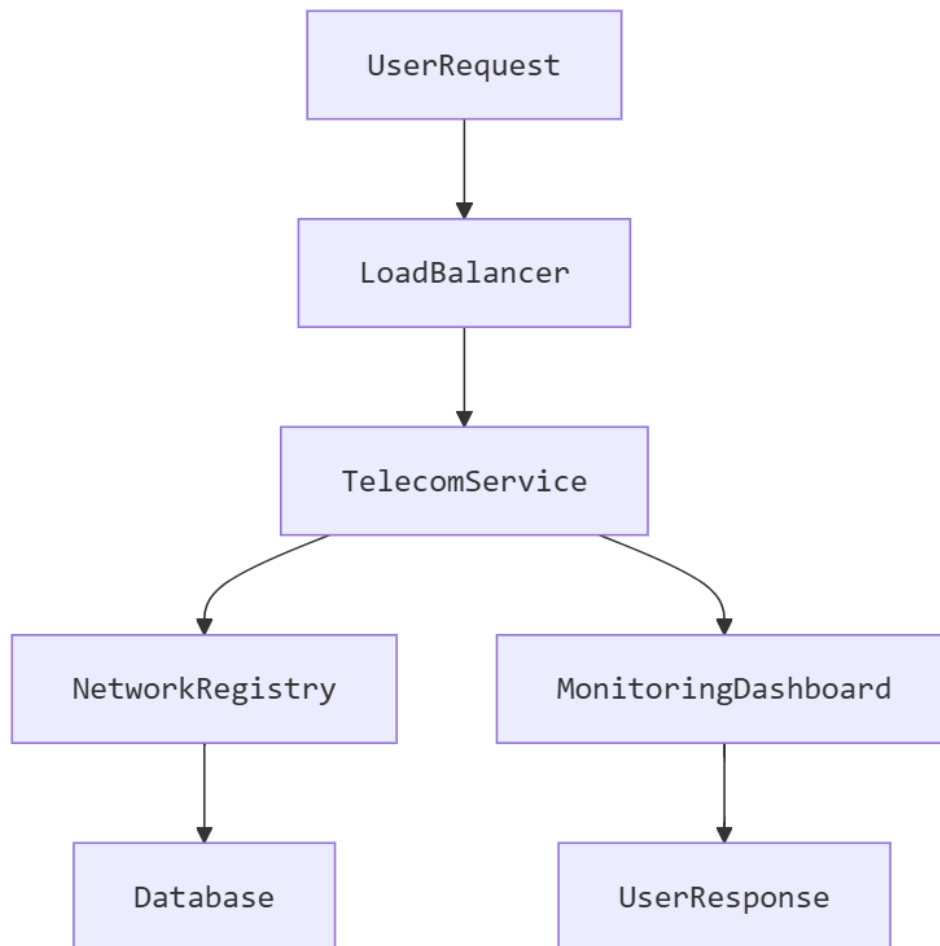
- Base Image: python:3.11-slim or alpine-based telecom binaries
- Exposed Ports: 5060 (SIP), 8080 (Dashboard), 3000 (Monitoring), etc.

## 2.2 Process Flow

### End-to-End Provisioning Workflow



## 2.3 Information Flow



## 2.4 Components Design

### Component 1 — Terraform (Infrastructure Provisioning)

| Property       | Value             |
|----------------|-------------------|
| Provider       | AWS               |
| Region         | ap-south-1        |
| Instance Type  | t3.small          |
| VPC            | Custom or Default |
| Security Group | SIP + HTTP + SSH  |
| AMI            | Amazon Linux 2    |

### Key Terraform Steps

- Initialize provider
- Create VPC + Subnets
- Create Security Groups
- Create EC2 Instances
- Output public/private IPs

### Component 2 — Ansible (Configuration Management)

| Configuration    | Value                                   |
|------------------|-----------------------------------------|
| Inventory Source | Terraform output                        |
| Playbooks        | install-docker.yml, deploy-services.yml |
| SSH Access       | Private key from GitHub Secrets         |

### Responsibilities

- Install Docker & dependencies
- Pull telecom service containers
- Start telecom network services
- Setup monitoring stack

### Component 3 — Local Docker Telecom Network

| Component        | Port | Purpose           |
|------------------|------|-------------------|
| SIP Handler      | 5060 | Call signalling   |
| SMS Router       | 8081 | Message routing   |
| Registry Service | 9000 | Subscriber DB     |
| Monitoring       | 3000 | Grafana/Streamlit |

Docker compose provisions:

- Telecom containers
- Shared overlay network
- Volume mounts
- Health checks

#### Component 4 — GitHub Actions (Automation Pipeline)

| Step | Action                        |
|------|-------------------------------|
| 1    | Checkout Repo                 |
| 2    | Install Terraform & Ansible   |
| 3    | Run Terraform Init/Plan/Apply |
| 4    | Parse EC2 outputs             |
| 5    | Run Ansible Playbooks         |
| 6    | Validate services are running |

#### Component 5 — AWS Infrastructure

| Component      | Details               |
|----------------|-----------------------|
| EC2            | t3.small, 2 instances |
| VPC            | 1 public subnet, IGW  |
| Security Group | SIP/HTTP/SSH          |
| IAM            | Least privilege role  |

### 2.5 Key Design Considerations

| API             | Method | Purpose              | Auth  |
|-----------------|--------|----------------------|-------|
| /call/initiate  | POST   | Simulate call setup  | Token |
| /sms/send       | POST   | Send SMS routing job | Token |
| /registry/query | GET    | Subscriber info      | None  |
| /health         | GET    | Service health       | None  |



Example Response:

200 OK

"status: healthy"

### 3 References

- Terraform Docs — [developer.hashicorp.com](https://developer.hashicorp.com/terraform)
- Ansible Docs — [docs.ansible.com](https://docs.ansible.com)
- Docker Docs — [docs.docker.com](https://docs.docker.com)
- AWS EKS/EC2 — [docs.aws.amazon.com](https://docs.aws.amazon.com)
- Telecom Standards — 3GPP TS Series

### 4 GitHub

<https://github.com/atisha224/IaC-Provisioning-for-Telecomm-System>